INFOGRAFIA TECNICA: COMO FUNCIONA EL BACKEND EN GO

Proyecto TiendaCocis (Shalom) | Arquitectura Hexagonal + Patron Singleton + Patron Strategy

GO + POSTGRES
camelCase 100%

1. FLUJO DE UNA PETICION HTTP EN LA ARQUITECTURA HEXAGONAL (5 PASOS)

1. CLIENTE WEB

Next.js React
services/api.js
`JSON camelCase`
Puerto :3000

→

2. ADAPT. PRIMARIO

adapters/inbound
`productHandler.go`
`inventoryHandler.go`
Recibe HTTP REST

→

3. NUCLEO (HEXAGONO)

ports/ + core/ + domain/
- Valida reglas negocio
- Aplica Strategy stock

`inventoryStrategy.go`

→

4. ADAPT. SECUNDARIO + DB

adapters/outbound (SQL)
`database.go (Singleton)`
`productRepository.go`
`inventoryRepository.go`
-> PostgreSQL Docker :5434

2. MAPA DE LAS 4 CAPAS DE LA ARQUITECTURA HEXAGONAL (internal/)

CAPA 1: DOMINIO PURO (internal/domain/)

- product.go: IdProducto, NombreProducto, PrecioUnidad, Categoria
- inventory.go: IdInventorio, IdProducto, Stock, StockMin
- inventoryStrategy.go: Interfaz InventoryStrategy y calculos

Regla de Oro: No importa librerias de SQL ni HTTP.

CAPA 2: PUERTOS / INTERFACES (internal/ports/)

- inbound.go (Puertos de Entrada - Driving Ports):
  `interface ProductService / interface InventoryService`
- outbound.go (Puertos de Salida - Driven Ports):
  `interface ProductRepository / InventoryRepository`

CAPA 3: CASOS DE USO / CORE (internal/core/)

- productService.go (ServicioProducto):
  `ListarProductos, CrearProducto, ActualizarProducto...`
- inventoryService.go (ServicioInventario):
  `ListarInventario, CrearInventario, ActualizarStock...`

CAPA 4: ADAPTADORES (internal/adapters/)

- Primarios HTTP (inbound/): ManejarProductos, ManejarInventario, responderJSON, responderError.
- Secundarios SQL (outbound/): ObtenerInstancia, GuardarProducto, ModificarInventario, BuscarInventario.

3. PATRONES DE DISENO IMPLEMENTADOS EN GO (SINGLETON + STRATEGY)

PATRON CREACIONAL: SINGLETON (database.go)

¿Que problema resuelve?
Evita abrir multiples conexiones pesadas a PostgreSQL en cada peticion HTTP que llega desde el frontend.

¿Como funciona en Go?
Usa sync.Once en ObtenerInstancia() para garantizar una sola instancia global segura ante concurrencia:

```
var instancia *DatabaseConnection
var unaVez sync.Once
func ObtenerInstancia() *DatabaseConnection {
  unaVez.Do(func() { ... }) // 1 sola vez
}
```

PATRON COMPORTAMIENTO: STRATEGY (domain/)

Interfaz: InventoryStrategy (inventoryStrategy.go)
`CalcularValor(stock, precioUnidad) | CalcularStock(stock)`

1. StandardStrategy ('estandar')
Valor real directo: stock * precioUnidad (100% en S/)

2. LiquidacionStrategy ('liquidacion')
Depreciacion del 30%: stock * (precioUnidad * 0.70)

3. VolumenStrategy ('volumen')
Si stock >= 20 uds: bonifica +10% stock y ajuste de valor

4. ENDPOINTS REST EXPUESTOS POR EL BACKEND (cmd/server/main.go - Puerto :8080)

`GET`	`/api/products`	-> listarProductos()	`GET`	`/api/inventory?estrategia=...`	-> listarInventario()
`POST`	`/api/products`	-> crearProducto()	`POST`	`/api/inventory`	-> crearInventario()
`PUT`	`/api/products/{id}`	-> actualizarProducto()	`PUT`	`/api/inventory/{id}`	-> actualizarStock()
`DELETE`	`/api/products/{id}`	-> eliminarProducto()	`DELETE`	`/api/inventory/{id}`	-> eliminarInventario()

CHECKLIST DE REQUISITOS: camelCase en Go | Funciones en espanol (1 a 3 palabras) | 2 CRUDs PostgreSQL Docker | Moneda S/